WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

The Essence of Computing

Lecture 01: computation as change of state, and a short history of computing tools

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 About this module

02 What is computation?

03 A short history of computing tools

04 Building a computer

05 Summary

01

About this module

From switches to a computer

This module shows how a computer is built, starting from nothing more than switches that are on or off. No prior knowledge of electronics is needed.

Part 1: the foundations

- numeral systems
- Boolean algebra and Boolean functions
- logic gates
- combinational and sequential circuits

Part 2: the computer

- an instruction set architecture
- an arithmetic logic unit, registers and RAM
- a stepper and a controller
- all together: the **DX8**, a working 8-bit computer

Continued in Digital Design and Computer Architecture II (Semester 3).

The road map

FOUNDATIONS

1. **The essence of computing** ← today
2. Numeral systems
3. Boolean algebra: laws and theorems
4. Boolean functions: canonical forms
5. Karnaugh maps
6. Logic gates and universal gate sets

COMBINATIONAL CIRCUITS

7. Multiplexers and demultiplexers
8. Encoders, priority encoders and decoders
9. Timing of combinational circuits

SEQUENTIAL CIRCUITS

10. Finite state machines: Moore and Mealy
11. The clock; SR latch and D latch
12. Flip-flops: D, T and JK
13. Counters
14. Timing: setup and hold times

BUILDING THE DX8

15. Instruction set architecture
16. Arithmetic: adders, two's complement
17. Logic and comparison
18. The arithmetic logic unit
19. Memory: registers, RAM, register file
20. Processor design
21. Stepper, data path and controller
22. Building and programming the DX8
23. Course review and exam

How the module works

- Self-paced: all materials are on Moodle.
- Each week: two lectures, a central exercise with worked examples, and a tutorial in which the solutions of the week's exercise sheet are discussed.
- You build and test the circuits of every unit in the **Logisim-evolution** circuit simulator. Circuit files for the examples in the book are provided.
- Verilog models of many of the circuits are there if you want to go further.

Activity	Hours
Lectures	34
Labs and tutorials	36
Independent study	130
Total	**200**

7 ECTS credits

Assessment

Component	Weight	Due	Outcomes
Quizzes	20%	throughout	LO1–LO5
Circuit-design project: the DX8 computer	40%	throughout	LO3–LO6
Final exam	40%	end of module	LO1–LO6

- In the project you build the DX8 computer, or an extension of it, in Logisim-evolution and run your own programs on it.
- Permitted support materials are provided in the exam; no other aids are allowed.
- Work submitted after the deadline is penalised.
- All submitted work must be your own: see the Code of Conduct.

What you will be able to do

LO1 Represent numbers in binary, octal and hexadecimal, and convert between them.

LO2 Manipulate Boolean expressions, derive sum-of-products and product-of-sums forms, and simplify them with Karnaugh maps.

LO3 Implement Boolean functions with logic gates; design and analyse combinational circuits, including their delays.

LO4 Model sequential behaviour with Moore and Mealy machines; build latches, flip-flops, registers and counters; explain setup and hold times.

LO5 Explain the Von Neumann architecture and the role of an instruction set; design an ALU, a register file and byte-addressable memory.

LO6 Explain fetch, decode, execute and load; compare single-cycle, multi-cycle and pipelined designs; build and program a multi-cycle 8-bit computer in a circuit simulator.

Reading

The book

A. Khudiyev, *A Student's Computer Architecture Book I*, 2026. Free online at alikhudiyev.com. Circuit files and Verilog models: DX8 course material.

`Today:  Chapter 1, The Essence of Computing`

Recommended

N. Nisan, S. Schocken, *The Elements of Computing Systems*, 2nd ed., MIT Press, 2021.

S. L. Harris, D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Elsevier, 2022.

D. A. Patterson, J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*.

C. Petzold, *Code: The Hidden Language of Computer Hardware and Software*, 2nd ed., Microsoft Press, 2022.

Getting help

Module leader	Dr. Ali Khudiyev
Email	ali.khudiyev@wosm.academy
Office hours	Monday–Friday, 10:00–17:00 (Baku time, UTC+4)
Course page	moodle.wosm.academy

02

What is computation?

Computing before computers

Charles Babbage's Difference Engine and Analytical Engine, from the 1800s, are often called the very first computing devices. But the history of computing goes back much further:

- before Babbage, the abacus;
- long before the abacus, a bone with notches cut into it, used as early as 35,000 BCE;
- and before any tool, people counting and calculating in their heads.

The same process can be carried out by a computer, a calculator, pen and paper, a notched bone, your fingers, or your imagination alone.

> Computing is not about the tools. It is about the process that the tools carry out.

Tools and essence

The tools

- make computation efficient: how fast it is, and how much energy it takes
- keep changing: notches, beads, gears, relays, vacuum tubes, transistors

The essence

- is the process itself
- stays the same, whatever the tool
- sets the rules that every tool has to follow

> **Remark**
>
> This module studies both: the essence (Boolean algebra, state machines) and one family of tools built on it (digital circuits).

Computation without people

Imagine looking at the Earth from space, with no people on it. You would still see

- the planet moving and the weather changing,
- trees growing, bees flying, animals fighting,
- waves colliding.

Each of these processes moves the world from one state into another.

> **The view of this module**
>
> Changing from one state to the next takes some energy. In computer science we see this change of state as the computation itself.

State and computation

> **Definition (State)**
>
> The *state* of a system is what we need to know about it at one moment to say, given the rules and the inputs, what it will do next.

> **Definition (Computation)**
>
> A *computation* is a sequence of changes of state, in which each next state follows from the current state, and the input, by fixed rules.

Neither definition mentions a tool: the same rules can be followed in your head, with pen and paper, or by a machine.

Example: adding with pen and paper

Compute 47 + 38, one column at a time, from the right.

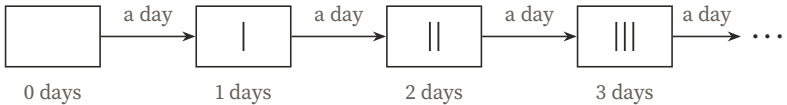
Step	Written so far	Carry
0	(nothing)	0
1	5	1
2	85	0

- State: the digits written so far, and the carry.
- Rule: add the two digits of the next column and the carry; write the last digit of the sum, carry the rest.
- Step 1: 7 + 8 + 0 = 15. Step 2: 4 + 3 + 1 = 8.

The adders we build later work the same way, in binary: column by column, passing a carry along (Unit 16).

The simplest computation: a tally

Count the days by cutting one more notch into a bone every day.



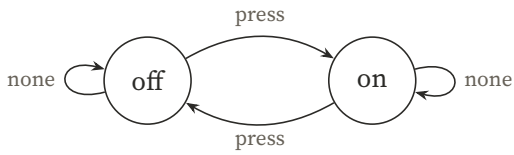
- State: the number of notches. Rule: one more notch per day.
- The same computation runs on a bone, on your fingers, in your head, or in a counter circuit `(Unit 13)`.

Current state, input, next state

The next state is a function of the current state and the input: $s_{t+1} = f(s_t, x_t)$.

A lamp with a push button

State s_t	Input x_t	Next s_{t+1}
off	none	off
off	press	on
on	none	on
on	press	off

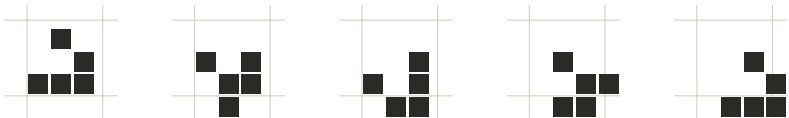


> **Coming back**
>
> Finitely many states and a rule like this make a finite state machine (Unit 10). The lamp's on/off memory, built from gates, is a T flip-flop (Unit 12).

Our logo is a computation

In Conway's *Game of Life* (1970) the state is a grid of cells, each alive or dead. At every step, all cells change at once, by looking at their 8 neighbours:

- a live cell with 2 or 3 live neighbours stays alive; every other live cell dies;
- a dead cell with exactly 3 live neighbours comes alive.



After 4 steps the *glider* has its original shape again, one cell further along the diagonal. Generation 0 is the mark on every one of these slides.

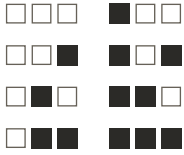
```
In Life, isolated cells starve and cells in company survive.
```

How many states?

A row of switches, each on or off. How many different states can the row be in?

Switches	States
1	2
2	4
3	8
n	2^n

The 8 states of 3 switches (filled = on)



Each extra switch doubles the number of states. Naming these states with numbers is the topic of the next lecture: numeral systems, and binary in particular (Unit 2).

03

A short history of computing tools

Counting by hand

c. 35,000 BCE	The **Lebombo bone**: an 8 cm piece of baboon fibula with notches cut into it, believed to have been used to count days and track time (its purpose is debated).
antiquity	The **abacus**: counters on lines or rods. Where a counter sits says what it is worth, an idea we meet again in positional numeral systems (Unit 2).
2nd–1st c. BCE	The **Antikythera mechanism**: bronze gear wheels that predict the positions of the Sun and the Moon, and eclipses.

The tools for computing may have started then. The essence of computing was there long before: people counted and calculated in their heads.

Machines that calculate

1642	Blaise Pascal's calculator adds and subtracts with gear wheels; a carry passes from one wheel to the next.
1670s–1690s	Gottfried Wilhelm Leibniz's stepped reckoner also multiplies and divides.
1703	Leibniz publishes his account of **binary** arithmetic: every number written with the digits 0 and 1 only.
1804	Joseph Marie Jacquard's loom: **punched cards** choose the pattern it weaves. Change the cards, and the same machine weaves something else.
1822	Charles Babbage proposes the **Difference Engine**.

Babbage’s engines

- The **Difference Engine** (1822) computes tables of polynomials automatically, using addition only.
- The **Analytical Engine** (designed from 1837) is general-purpose. It is programmed with punched cards, as Jacquard’s loom is, and has a *mill* that calculates and a *store* that holds numbers: today we say processor and memory.
- In 1843 Ada Lovelace publishes notes with a method for computing Bernoulli numbers on the Analytical Engine, often called the first computer program.

> **Remark**
>
> Neither engine was finished in Babbage’s lifetime. In 1991 the Science Museum in London completed his Difference Engine No. 2 from his drawings. It works.

The method of differences

How can a machine that can only add compute n^2?

n	0	1	2	3	4	5
n^2	0	1	4	9	16	25
first diff.	1	3	5	7	9	
second diff.	2	2	2	2		

The second differences are always 2. Work upwards from them: $7 + 2 = 9$, then $16 + 9 = 25$. Two additions per new value, and no multiplication.

The method of differences as a computation

- State: the pair (v, d), the current value and its difference.
- Rule: $v \leftarrow v + d$, then $d \leftarrow d + 2$.

$$(0,1) \to (1,3) \to (4,5) \to (9,7) \to (16,9) \to (25,11) \to \cdots$$

> **Remark**
>
> A polynomial of degree k has constant k-th differences, so k additions per step are enough. Difference Engine No. 2 works with seven orders of differences on 31-digit numbers.

From gears to switches

1854 George Boole, *An Investigation of the Laws of Thought*: an algebra of true and false (Unit 3).

1936 Alan Turing defines computation precisely: a machine in one of finitely many states reads and writes symbols on a tape, following fixed rules.

1937 Claude Shannon shows that Boolean algebra describes circuits of relays and switches (Unit 3–6).

1945 ENIAC: electronic, with about 18,000 vacuum tubes. John von Neumann's report on the EDVAC: program and data in the same memory (Unit 15).

From switches to chips

1947	The **transistor** (Bell Labs): a small, fast and reliable electronic switch.
1958–59	The **integrated circuit** (Jack Kilby, Robert Noyce): many transistors on a single chip.
1971	The Intel 4004, the first commercial **microprocessor**: a 4-bit processor on one chip, with about 2,300 transistors.
today	A phone processor has billions of transistors.
this module	The **DX8**: an 8-bit computer that you build yourself.

You may be reading this on a computer, a phone or even a smart watch. Imagine how that would have looked to the people who cut notches into bones.

Same essence, better tools

fingers → notched bone → abacus → gears → punched cards → relays → vacuum tubes → transistors → chips

- The tools became faster, smaller and cheaper, and use less energy: they became more efficient.
- The essence did not change: a state, and rules that change it.
- From Boole and Shannon on, the rules are Boolean algebra and the tool is a switch that is on or off. This whole module is built on that.

> We have come a long way. The task now is to understand this long history, and to push it further than where it stands today.

04

Building a computer

Layers of abstraction

The DX8, running your programs	22
The processor: stepper and controller	20–21
ALU, registers and RAM	15–19
Sequential circuits	10–14
Combinational circuits	7–9
Logic gates	6
Boolean algebra and Boolean functions	3–5
Numbers in binary	2
Switches: on or off	1

- Each layer is built only from the layer below it.
- Each layer hides the details of the layer below from the layer above.
- This is how a system with millions of parts stays understandable.

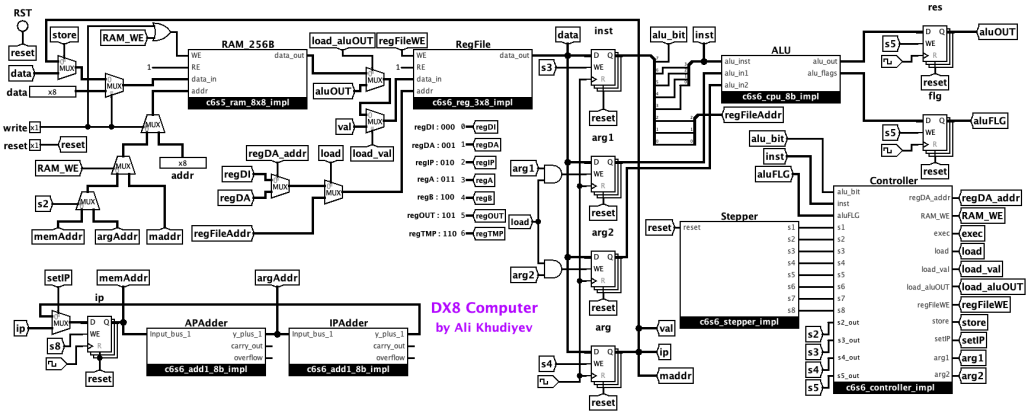
```
Numbers on the right:  units
```

What does it take to build a computer?

About 70 pages of the book to learn the fundamentals, and then about 15 pages to put everything together.

Chapter		Units
1	The Essence of Computing	1
2	Foundations & Boolean Algebra	2–5
3	Logic Gates & Circuits	6
4	Combinational Logic	7–9
5	Sequential Logic	10–14
6	Let's Build a Computer!	15–22
7	Epilogue	

Where we are going: the DX8



The DX8 computer in Logisim-evolution, from Chapter 6 of the book.

The DX8 at a glance

- an 8-bit computer
- instructions of 16 bits (2 bytes)
- a register file of 8 registers, selected by 3-bit addresses
- $2^8 = 256$ bytes of RAM
- an arithmetic logic unit (ALU)
- a stepper and a controller that drive every step, in time with the clock
- each instruction runs in four stages: fetch, decode, execute and load

Every part of it is made of the circuits of Units 6–14, which are made of logic gates, which are made of switches.

05

Summary

Summary

- Computation is a change of state that follows rules. It does not depend on the tool that carries it out.
- The state, the input and a rule give the next state: $s_{t+1} = f(s_t, x_t)$.
- Tools make computation efficient. From notched bones to chips with billions of transistors, the tools changed; the essence did not.
- Boolean algebra and on/off switches are the foundation of this module.
- We will build the DX8 computer layer by layer, from switches up.

Before the next lecture

- Read Chapter 1 of the book, *The Essence of Computing*.
- Install Logisim-evolution (free), the circuit simulator for the exercises.
- Think of three processes around you. For each: what is the state, what changes it, and which tool, if any, carries it out?

> **Next lecture** (Unit 2)
>
> Numeral systems: additive and positional systems; binary, octal and hexadecimal.